1. Sesión 7

La verdad esta guía me confundió al principio. Yo venía de la guía 6 donde solo hacía los comandos en MySQL, y en las anteriores solo hacía Java solito, sin base de datos. Y ahora me pedían juntar los dos.

Lo primero que pensé fue: "esto lo tengo que hacer en htdocs", porque como uso XAMPP siempre pensaba que todo iba ahí. Me puse a buscar dónde crear la carpeta en htdocs y no me cuadraba nada, hasta que me di cuenta que eso es solo para PHP. Java no se corre desde ahí.

Cuando entendí eso, todo se hizo más fácil. Esta guía era solo hacer que mi programa de Java, que ya sabía hacer, se conecte de verdad a la base de datos `gamestore_db` que ya tenía creada, y que cuando yo le ponga "registrar estudiante" en Java, de verdad se guarde en MySQL y no solo se imprima en pantalla.

Lo más difícil no fue el código en sí, fue hacer que se conecten. Porque mi código estaba bien, pero me salía error de driver y no entendía por qué. Ahí aprendí que Java necesita un archivo extra, el Connector/J, que es como el traductor entre Java y MySQL. Sin eso, no hay conexión por más que todo lo demás esté bien.

2. Qué usé y cómo preparé todo

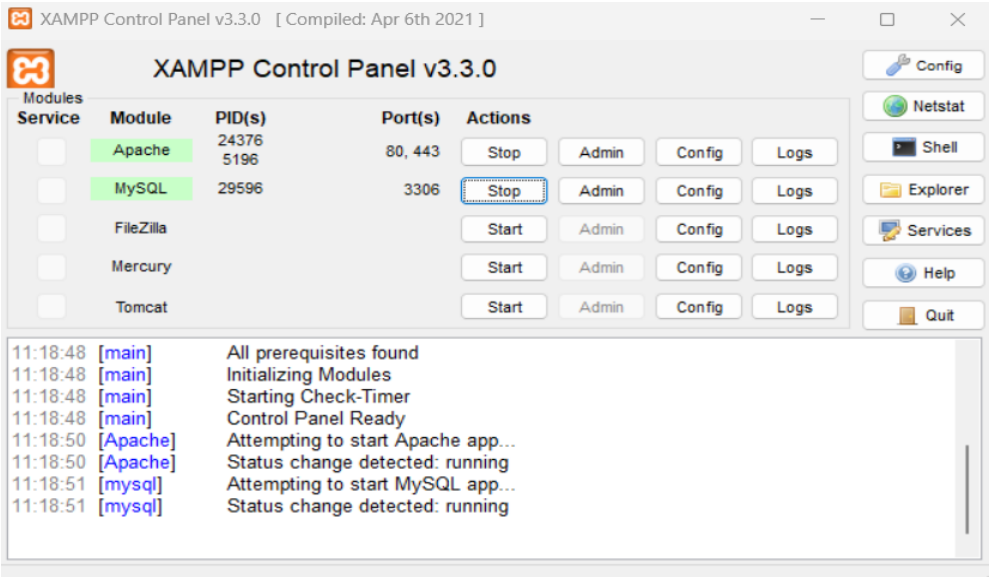
Para esta guía usé VS Code con JDK 17, igual que las guías anteriores, y XAMPP solo para prender el MySQL. No usé htdocs para nada.

Primero abrí XAMPP y le di Start a MySQL. Lo dejé corriendo en localhost:3306. Apache no lo necesité, porque no era página web.

Después hice mi carpeta de proyecto en:

`C:\GameStoreEscolarFer`

Adentro creé dos carpetas: `src` para los archivos .java y `lib` para el conector.



3. El problema del driver y cómo lo arreglé

Cuando corrí por primera vez el programa me salió esto en la terminal:

`No suitable driver found for jdbc:mysql://localhost:3306/gamestore_db`

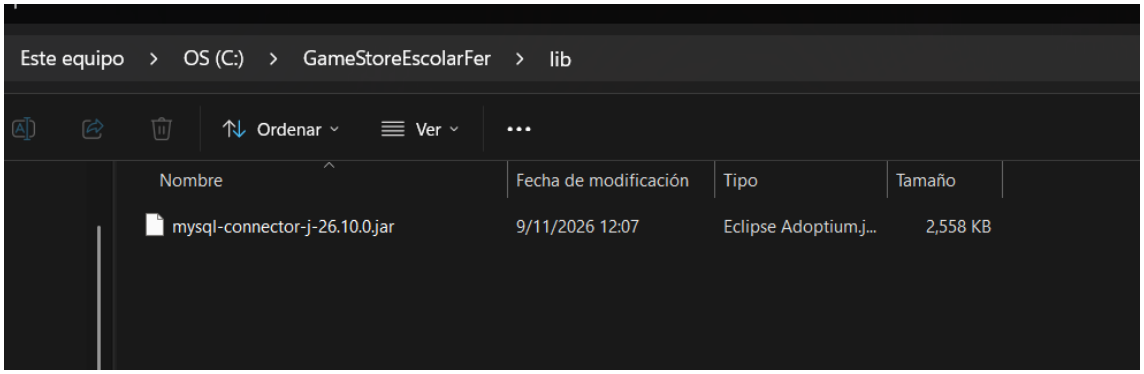
Y después otro error que decía que la conexión era null.

Yo pensé que mi código estaba mal, pero no. Lo que pasaba era que a Java le faltaba el driver. Sin ese archivo .jar, Java no sabe cómo hablar con MySQL.

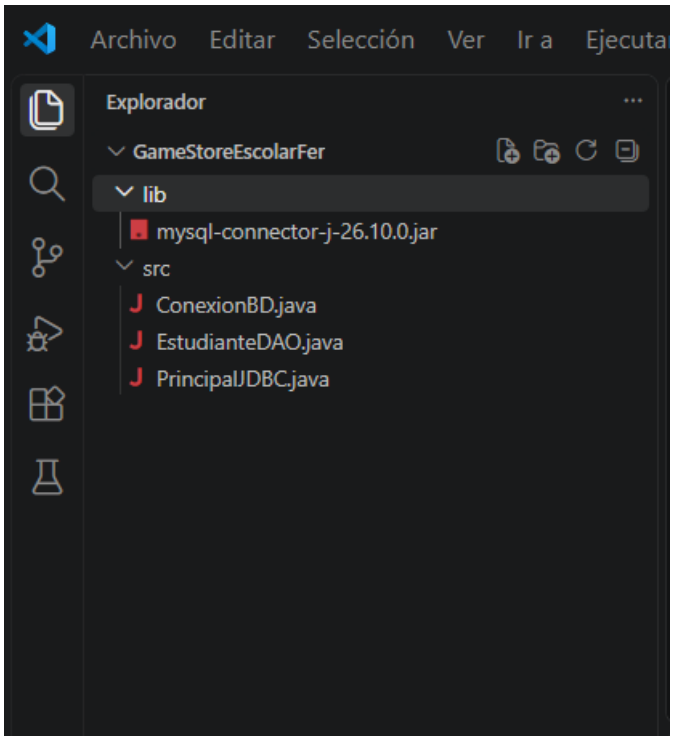
Para arreglarlo hice esto paso a paso:

Primero entré a la página de MySQL Connector/J. Ahí donde decía Select Operating System elegí "Independiente de la plataforma", no Windows ni Ubuntu. Después elegí el que decía ZIP Archive y le di Download. Me salió una página para iniciar sesión con cuenta de Oracle, pero no me registré, le di abajo donde decía "No, gracias, simplemente inicia la descarga" y se descargó solo.

Cuando se descargó, lo descomprimí. Adentro había varios archivos, pero el único que me importaba era el que terminaba en .jar. Ese lo copié y lo pegué en mi carpeta `lib`.



Después en VS Code fui abajo a JAVA PROJECTS, en Referenced Libraries le di al +, seleccioné el .jar que había pegado en lib y ya quedó agregado. Recién ahí VS Code lo empezó a usar.



4. Cómo hice el procedimiento paso a paso

Trabajo 1 - La clase ConexionBD.java

Esta clase es la que se encarga de conectarse. La hice centralizada para no estar repitiendo la conexión en cada archivo. Si mañana cambio la contraseña, solo la cambio aquí y ya.

Lo que hice fue poner tres datos: la URL, el usuario y la contraseña. La URL es la que apunta a mi base `gamestore_db` en localhost:3306, el usuario es `root` y la contraseña en mi caso está vacía porque así lo tengo en XAMPP.

Después hice un método que se llama `obtenerConexion()` que usa `DriverManager.getConnection`. Si se conecta bien, imprime "¡Conexión exitosa!". Si falla, imprime el error. Esto lo puse con try-catch para que no se caiga feo el programa.



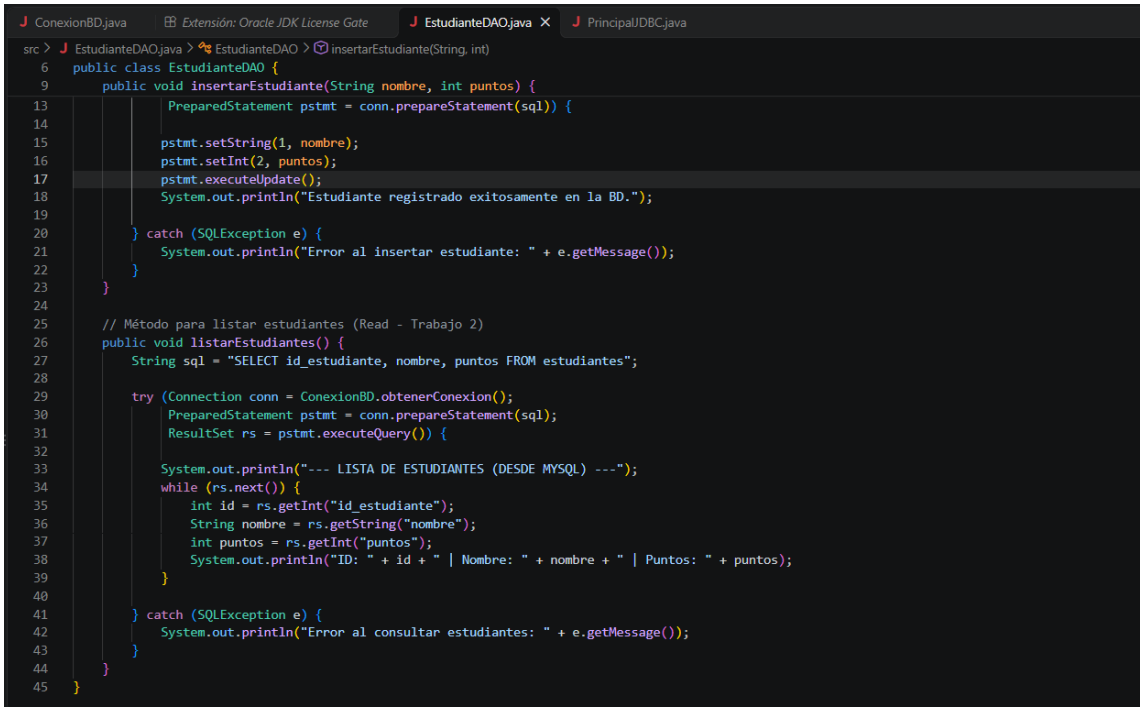
Trabajo 2 - La clase EstudianteDAO.java

Esta es la que hace el CRUD. DAO significa que es la que se encarga de hablar con la base de datos.

Primero hice el método para insertar estudiante. Ahí no usé Statement normal, usé PreparedStatement. El SQL lo puse con `?` en vez de poner los datos directo. Los `?` son los espacios en blanco donde después van los datos. Después con `setString` le pongo el nombre en el primer `?` y con `setInt` le pongo los puntos en el segundo. Y al final con `executeUpdate()` lo guarda.

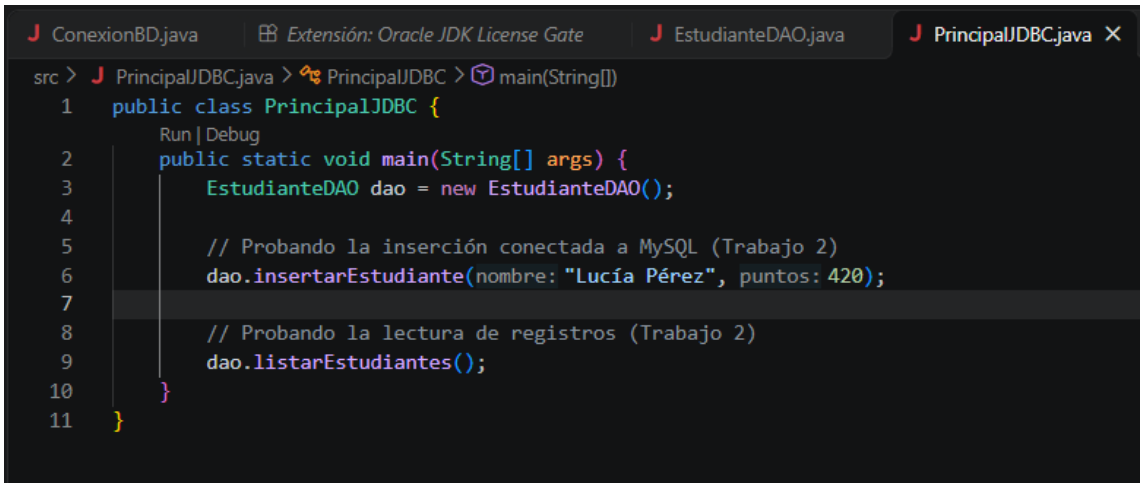
Después hice el método para listar estudiantes. Ahí el SQL es un SELECT de id, nombre y puntos. Lo ejecuté con `executeQuery()` y eso me devuelve un ResultSet, que es como una tablita con los resultados. Con un `while` fui recorriendo fila por fila y con `getInt` y `getString` fui sacando cada dato para imprimirlo.

En los dos métodos usé try-with-resources, que es poner la Connection y el PreparedStatement adentro del paréntesis del try, para que Java los cierre solo al final, aunque haya error. Eso es lo que pedía la guía, para no dejar conexiones abiertas.



Clase PrincipalJDBC.java para probar

Esta es la clase main para probar que todo funciona. Ahí creé un DAO, después llamé a insertar estudiante con "Lucía Pérez" y 420 puntos, y después llamé a listar estudiantes para ver si se había guardado.



5. Cómo lo ejecuté y qué me salió

Primero me aseguré de que MySQL esté prendido en XAMPP y que la base gamestore_db ya exista de la guía 6, con Carlos Mendoza y Ana Torres adentro.

Después le di Run a PrincipalJDBC.java y me salió esto en la terminal:

¡Conexión exitosa a la base de datos MySQL!

Estudiante registrado exitosamente en la BD.

¡Conexión exitosa a la base de datos MySQL!

LISTA DE ESTUDIANTES (DESDE MYSQL)

ID: 1 | Nombre: Carlos Mendoza | Puntos: 600

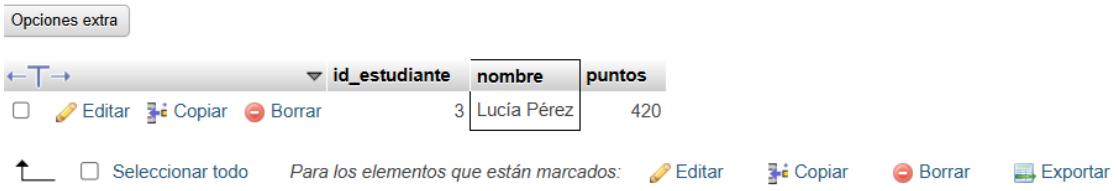
ID: 2 | Nombre: Ana Torres | Puntos: 350

ID: 3 | Nombre: Lucía Pérez | Puntos: 420

Me salió el mensaje de "Conexión exitosa" dos veces y al principio pensé que era un error, pero no. Es porque insertar y listar abren cada uno su propia conexión con try-with-resources, entonces cada uno se conecta y se desconecta por su cuenta. Y Lucía Pérez me salió con ID 3 porque Carlos y Ana ya eran 1 y 2, y el ID se pone solo con AUTO_INCREMENT.



Después entré a phpMyAdmin para comprobar que de verdad se había guardado y no solo se había impreso. Y sí, ahí estaba Lucía Pérez en la tabla estudiantes.



6. Conclusión

Con esta guía entendí para qué sirve JDBC. JDBC es como un adaptador: Java habla un idioma y MySQL habla otro, y el Connector/J es el adaptador que los hace entenderse. DriverManager es el que elige el adaptador correcto según la URL.

También entendí por qué se usa PreparedStatement con `?` en vez de pegar los datos directo en el SQL. Es como un formulario ya impreso, donde solo rellenas los espacios. Así nadie puede meter código SQL malicioso por un

campo de texto, que eso es la inyección SQL. Y también aprendí que cada `?` tiene que tener su `setXxx()`, porque si me falta uno, me sale una SQLException y no se ejecuta.

Al principio me frustré por lo del driver, pero ya aprendí que en Java siempre hay que agregar el .jar del conector a las librerías, si no, no hay conexión por más que el código esté bien.